

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Escuela Nacional de Estudios Superiores, Unidad Juriquilla

Temas Selectos de Ciencia de Datos I

Guía de ejercicios: Tiempos de traslado y arreglos NumPy

Tema 1 | Subtema 1.4: Introducción a NumPy

Material de trabajo con datos sintéticos. No describe movilidad real ni permite inferencias sobre una población.

Propósito

Resolver un caso continuo en dos sesiones de 120 minutos. Al terminar, cada equipo entregará un notebook ejecutable y explicaciones breves que demuestren que puede leer la estructura de un ndarray, seleccionar datos con seguridad y resumirlos mediante operaciones vectorizadas.

Sesión	Foco	Evidencia
1. Estructura y selección	ndarray, shape, dtype, indexación, slicing, vistas y copias	Respuestas justificadas y celdas ejecutables
2. Operación y resumen	Vectorización, máscaras, ufunc y agregaciones por eje	Notebook final con interpretaciones y unidades

Requisitos

- Trabajar en equipos de dos o tres. Cada integrante debe poder explicar una celda elegida al azar.

- Usar Python 3, NumPy y un Jupyter Notebook local. Iniciar con `import numpy as np`.
- Conservar las unidades: minutos, segundos o minutos acumulados, según corresponda.
- No usar pandas, archivos CSV, datos reales ni funciones no revisadas en clase.
- Antes de ejecutar, escribir una predicción para los incisos marcados con **P**.

El Caso

Un equipo de movilidad universitaria quiere organizar mediciones de duración de traslado. Durante cinco días se registraron tres rutas; cada celda contiene minutos. El conjunto es deliberadamente pequeño para que se pueda razonar sobre su forma, sus selecciones y sus agregaciones.

Trabajaremos con esta matriz durante las dos sesiones:

```
viajes = np.array([[32, 41, 28],
                  [37, 46, 35],
                  [29, 44, 51],
                  [42, 39, 33],
                  [48, 52, 36]])
```

Día (fila)	Ruta A	Ruta B	Ruta C
Lunes	32	41	28
Martes	37	46	35
Miércoles	29	44	51
Jueves	42	39	33
Viernes	48	52	36

Convención de lectura: eje 0 = días/filas; eje 1 = rutas/columnas. Los nombres son una leyenda humana: el ndarray contiene solo valores numéricos.

Preparación

Cree un notebook llamado `apellido1_apellido2_caso_numpy.ipynb`. Importe NumPy, defina `viajes` exactamente como se indica en el caso y ejecute la celda. Registre la versión de NumPy en un comentario.

Ejercicio 1. Leer la estructura

- **P.** Antes de ejecutar código, indique ndim, shape y el tipo general esperado de viajes.
- Escriba una celda que verifique ndim, shape y dtype. Explique en una frase qué representa cada componente de shape.
- Indique qué eje corresponde a los días y cuál a las rutas. No responda solo con 'filas' o 'columnas': relacione cada eje con el contexto.

Ejercicio 2. Crear e inspeccionar arreglos

- Cree indices_dias con los enteros de 0 a 4 usando np.arange. Registre su shape y dtype.
- Cree conteos con ceros y forma (5, 3), y referencia con todos los valores iguales a 30 y forma (5, 3). Explique una posible finalidad de cada arreglo en este caso.
- Convierta np.array([3.7, -1.2]) a entero con astype(np.int32). Indique los valores obtenidos y por qué esta conversión podría ser riesgosa para una medición.

Ejercicio 3. Coordenadas y ventanas

- **P.** Prediga valor, shape y significado contextual de viajes[2, 1]. Verifique después en el notebook.
- **P.** Prediga salida y shape de viajes[:, 0]. Explique por qué el resultado ya no es una matriz 5 x 3.
- **P.** Obtenga las primeras dos filas y las rutas B y C con una sola expresión de slicing. Dibuje o describa la región seleccionada y reporte su shape.
- Seleccione las filas en orden viernes, lunes y miércoles mediante indexación avanzada. Reporte la forma y explique por qué este orden no es un rango continuo.

Ejercicio 4. Una modificación, dos comportamientos

- Cree bloque = viajes[:, 2]. Cambie bloque[0, 0] a 999. Muestre bloque y viajes. Explique el cambio observado y restaure viajes a su valor original antes del siguiente inciso.
- Cree seleccion = viajes[viajes > 45]. Cambie seleccion[0] a 999. Muestre seleccion y viajes. Explique por qué ahora el origen no cambia.
- Escriba una expresión que produzca una selección independiente de las primeras dos filas y dos columnas. Compruébela con una mutación controlada.

Ejercicio 5. Ficha de lectura

Expresión	Qué selecciona	Shape	Ejes / contexto	Vista o copia
viajes[3, 2]				
viajes[1:4, 1:]				
viajes[viajes > 45]				
viajes[[4, 0, 2]]				

Cierre de sesión 1: entregue el enlace o archivo del notebook y una frase que complete: '*Uso .copy() cuando...*'.

Sesión 2: Vectorización y agregaciones

Tiempo	Actividad	Producto
0-15 min	Diagnóstico y recuperación de ejes	Predicción de shapes
15-45 min	Ejercicios 6 y 7	Transformaciones y máscara
45-75 min	Ejercicio 8	Agregaciones interpretadas
75-95 min	Ejercicio 9	Comparación con ciclo
95-120 min	Reto integrador	Notebook y respuesta final

Antes de empezar

Reinicie el kernel y ejecute el notebook desde la primera celda. Restaure viajes con la matriz original; no continúe si conserva algún 999 del ejercicio de vistas y copias.

Ejercicio 6. Operaciones sobre todo el arreglo

- **P.** Cree segundos = viajes * 60. Prediga su shape y unidad. Verifique y explique por qué no se requirió un ciclo explícito.
- Cree diferencia = viajes - 30. Identifique qué significan valores negativos, cero y positivos. No los interprete como calidad buena o mala.
- Cree exceso = np.maximum(viajes - 30, 0). Explique en una frase qué representa un cero en exceso.

Ejercicio 7. Condiciones y máscara

- **P.** Construya tardios = viajes > 45. Reporte su shape, dtype y cantidad de valores True usando tardios.sum().
- Obtenga los valores de viajes que cumplen la condición. Explique la diferencia entre la máscara y la selección resultante.
- Construya la condición de viajes mayores que 40 y menores o iguales a 50. Use paréntesis y el operador &. Reporte los valores seleccionados.

Ejercicio 8. Reducir un eje sin perder el contexto

- Calcule viajes.sum(), viajes.mean(), viajes.min() y viajes.max(). Para cada resultado, escriba su unidad y una frase de interpretación prudente.
- **P.** Calcule promedio_por_ruta = viajes.mean(axis=0). Prediga su shape antes de ejecutar. Asocie cada resultado con Ruta A, B y C.
- **P.** Calcule maximo_por_dia = viajes.max(axis=1). Prediga su shape antes de ejecutar. Asocie cada resultado con Lunes a Viernes.
- Explique con sus palabras la frase: *'el eje nombrado es el eje que se reduce'*. Use uno de los dos resultados anteriores como evidencia.

Ejercicio 9. Dos expresiones, un resultado

Complete y ejecute el ciclo; luego compare su resultado con segundos. No se pide medir tiempo.

```
segundos_ciclo = np.zeros_like(viajes)
for i in range(viajes.shape[0]):
    for j in range(viajes.shape[1]):
        segundos_ciclo[i, j] = _____
```

- Verifique la igualdad con np.array_equal(segundos_ciclo, segundos).

- Escriba dos diferencias entre ambas soluciones: una sobre claridad y otra sobre las decisiones manuales que exige el ciclo.

Reto integrador: Informe breve para el equipo

En una sección final del notebook, produzca las siguientes evidencias y un informe de 120 a 180 palabras:

- Definición reproducible de viajes y una verificación de shape y dtype.
- Transformación a segundos y exceso respecto de 30 minutos, con shape y unidad.
- Máscara de viajes mayores que 45, cantidad de casos y valores seleccionados.
- Promedio por ruta y máximo por día, ambos correctamente etiquetados e interpretados.
- Una limitación: explique por qué este ejercicio no permite concluir nada sobre la movilidad real ni establecer causas.

Criterios de entrega

Criterio	Puntos	Se observa cuando...
Estructura y selección	4	Lee forma, ejes y mecanismos de selección sin confundir vista y copia.
Código reproducible	4	El notebook corre desde un kernel limpio y usa NumPy de forma correcta.
Vectorización y máscara	4	Expresa transformaciones y condiciones sobre el arreglo completo.
Agregaciones e interpretación	5	Relaciona axis, shape, etiquetas y unidades de manera correcta.
Comunicación y límites	3	Explica resultados con precisión y declara una limitación del caso sintético.

Referencias de apoyo

- McKinney, W. (2022). *Python for Data Analysis* (3a ed.), cap. 4, secciones 4.1, 4.3 y 4.4. <https://wesmckinney.com/book/numpy-basics.html> (consulta: 30 de agosto de 2026).
- UNAM, ENES Juriquilla. Programa de estudios y planeación didáctica de Temas Selectos de Ciencia de Datos I, Tema 1, subtema 1.4. Fuentes institucionales locales.